SENIOR FRONTEND TAKE-HOME · PROJECT REPORT

OrderDesk

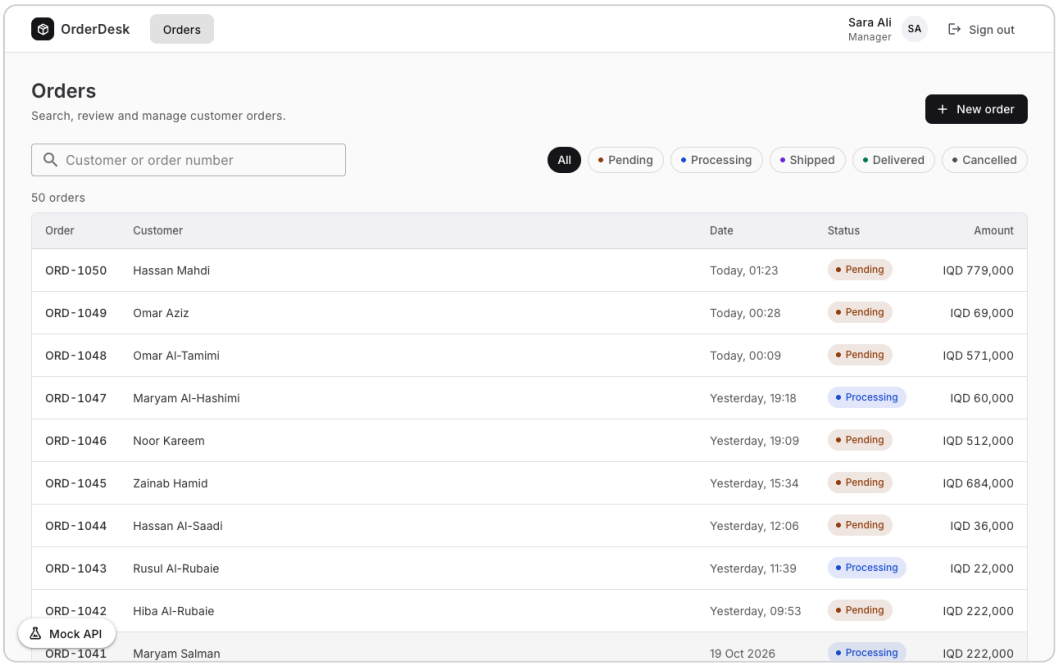
An order-management screen in Vue 3, the Composition API and TypeScript, running against a mock API built with axios-mock-adapter. This report shows what was built and the test results behind each requirement in the brief.

73/73
tests passing

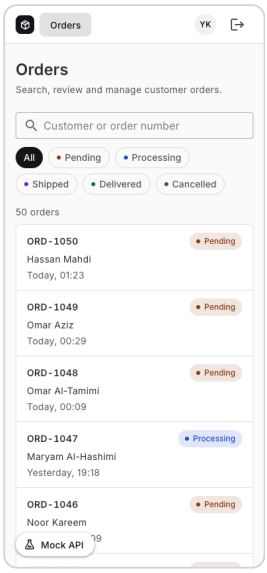
31
brief requirements mapped

41
tests cited as evidence

12.3 s
full test run



Orders list, signed in as Manager



Phone, Support: no amounts

QUALITY GATES

CHECK	COMMAND	RESULT
Tests	`npm test`	73 passed · 0 failed
Lint	`oxlint` + `eslint`	0 problems
Formatting	`prettier --check`	clean
Type-check and build	`npm run build`	passed

How it works

ARCHITECTURE

- **Server state** lives in TanStack Vue Query, one cache entry per search, so a slow older response can never replace newer results.
- **The session** (token, user, permissions) is the only Pinia store.
- **Every request** goes through one axios instance whose interceptors attach the token and turn failures into one typed `ApiError`.
- **Forms** run on TanStack Form and validate with the zod schema the mock uses, so 422 errors land under the matching fields.
- **The UI** is Vuetify 4. Its theme and component defaults define every shared style once.
- **Loaders sit on what's loading:** skeletons shaped like the screen they replace, a bar on the results while a search or filter updates them.
- **Filters and the open panel** live in the URL, so refresh, Back and shared links work.

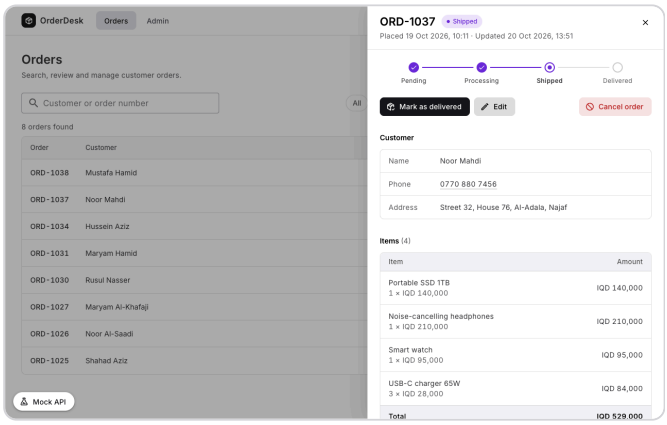
PERMISSION MODEL

- The UI checks **permissions, never roles**: `session.can('orders.cancel')`. Roles exist only on the server.
- **No permission → hidden.** Allowed, but blocked by the order's state → **disabled, with the reason** on screen.
- The mock enforces every rule again: **401** ends the session, **403** is explained and `/me` re-read, **409** for final orders, **422** per field.
- One rule beyond the brief: creating or editing also needs `orders.view_amount`, since orders carry prices.

ROLES, AS SERVED BY GET /ME

PERMISSION	ADMIN	MANAGER	SUPPORT
`orders.view`	✓	✓	✓
`orders.view_amount`	✓	✓	—
`orders.create`	✓	✓	—
`orders.update`	✓	✓	—
`orders.update_status`	✓	✓	—
`orders.cancel`	✓	—	—
`admin.access`	✓	—	—

DETAILS PANEL



Admin view: progress line, next step, edit and cancel

Requirements and evidence

Each requirement from the brief, with the passing tests that prove it. Every test cited here was matched against the Vitest results of the run on the last page.

REQUIREMENT	EVIDENCE
LOGIN	
Login page with email and password	✓ checks the form before calling the API, then explains wrong credentials auth
Three test users: Admin, Manager, Support	✓ returns a token for valid credentials, 401 for wrong ones and 422 for missing ones mock-api ✓ fills in a demo account in one click auth
Role and permissions loaded from GET /me	✓ returns the current user with their role and full permission list from /me mock-api
Refreshing keeps the user logged in	✓ keeps the user signed in after a reload auth
Logged-out users go to login; a 401 logs out	✓ sends visitors to sign in, then back to the page they asked for auth ✓ signs the user out when the API rejects their session, then brings them back auth
Switching users leaves nothing behind	✓ leaves nothing from the previous user behind when switching accounts auth ✓ drops responses that arrive after the user who asked for them signed out auth
ROLES AND PERMISSIONS	
Admin page can’t be reached by URL without permission	✓ is hidden from manager users and refuses them when they type its URL permissions ✓ is hidden from support users and refuses them when they type its URL permissions
Mock returns 403 for forbidden actions	✓ returns 403 when support creates an order mock-api ✓ returns 403 when manager cancels an order mock-api
Amounts are never sent to Support	✓ never sends order amounts to users without orders.view_amount mock-api ✓ are hidden from support users, who never receive them permissions
UI handles a 403, not just hidden buttons	✓ don't rely on hidden buttons: a 403 is explained and permissions re-checked permissions
Delivered and cancelled orders are final, even for Admin	✓ rejects every change to a delivered order, even from an admin mock-api ✓ rejects every change to a cancelled order, even from an admin mock-api ✓ keep delivered orders locked, even for admins, and say why permissions
Hidden vs disabled, explained	Missing permission → hidden. Blocked by order state → disabled with the reason. See README. ✓ keep delivered orders locked, even for admins, and say why permissions
Easy to add roles and permissions	The UI checks permissions, never roles. A new role is a server-only change.
MOCK API	
About 50 generated orders	50 deterministic orders from a fixed seed, kept in localStorage.
POST /auth/login, 401 on wrong credentials	✓ returns a token for valid credentials, 401 for wrong ones and 422 for missing ones mock-api

GET /orders?q=&status= filtered by the mock	✓ searches by customer or order number and filters by status on the server mock-api
GET /orders/:id with line items	✓ opens from the keyboard, shows the whole order and closes with Escape orders
POST / PUT orders, 422 per invalid field	✓ returns 422 with a message for each invalid field mock-api ✓ creates pending orders and computes the amount itself, ignoring client-sent values mock-api
PATCH status and POST cancel, rules on every write	✓ only moves an order one step forward and never cancels through a status change mock-api ✓ returns 403 when support changes a status mock-api
User from the Authorization token; 401 if missing or invalid	✓ identifies the caller from the token and rejects missing or unknown tokens mock-api
Random 300–1000 ms delay	On every response, errors included. Tests run at 0 ms so they stay fast and deterministic.
Easy way to force errors	“Mock API” panel: 500, network, 401, 403 or 422 on the next order request, also with an order's panel open. ✓ explains a failed load and recovers with "Try again" orders

SCREEN

Number, customer, amount, date, status	✓ are shown to managers permissions
Search and status filter; fast typing is safe	✓ searches and filters on the server, keeping both in the URL orders ✓ shows the latest results even when an earlier search responds last orders
Details in a side panel	✓ opens straight from a link, on top of the filtered list orders ✓ shows how far the order has come orders
Status change and cancel, by permission	✓ let managers walk an order through the workflow, but not cancel it permissions ✓ let admins cancel an order, after confirming permissions
Loading, no results and errors with retry	First loads show skeletons; new filters keep the old results and say what's coming. ✓ says which orders are on their way while new filters load orders ✓ explains when nothing matches and clears the filters in one click orders ✓ explains a failed load and recovers with "Try again" orders ✓ shows the skeleton, not another filter's orders, while a failed load is retried orders
Mobile and full keyboard use	Also checked in real Chrome at 1280 px, 900 px and 375 px. ✓ lists orders as cards that open with the keyboard and get focus back orders ✓ opens from the keyboard, shows the whole order and closes with Escape orders

CREATE AND EDIT

One form for create and edit, with line items and a live total	✓ checks the form, then creates the order and shows it in the list and details orders ✓ edits an order and shows the change in the details and the list orders
Client validation, and 422 errors under the fields	✓ shows the API's 422 errors under the matching fields orders
List and details show the saved data	✓ edits an order and shows the change in the details and the list orders

Test results

The UI tests render the whole app against the mock API and act like a user, using Vitest and Testing Library; the contract tests call the mock API directly. The brief asks for at least two tests, one of them about permissions.

SUITE	COVERS	TESTS	PASSED	TIME
`mock-api.test.ts`	The API contract: 401, 403, 409, 422, amounts	16	**16**	0.1 s
`auth.test.ts`	Sign-in, sessions, sign-out, stale responses	12	**12**	2.7 s
`permissions.test.ts`	What each role sees and can do	17	**17**	2.5 s
`orders.test.ts`	List, search, loading, details panel, create and edit	28	**28**	10.8 s
Total		**73**	**73**	**12.3 s**

BEYOND THE AUTOMATED TESTS

- Every flow was also exercised in real Chrome with Playwright scripts, at 1280 px and 375 px, for each role.
- Two review rounds with parallel reviewers (requirements, correctness, security, accessibility and UX, code quality), each finding checked by an adversarial verifier. Confirmed issues were fixed, with a regression test where the fix could be tested.
- A UI pass went through every loading, empty and error state in Chrome at 1280, 900 and 375 px, against the dev server and the production build. What it found was fixed, again with a regression test where the fix could be tested.

REPRODUCE

`npm install` `npm test` `npm run lint` `npm run build`

Measured on 21 October 2026 at commit 84819f3 · Node v24.22.1 · Vitest 4.1.11 · 19 test groups across 4 files.